

Design and Verification of a 4-Bit Processor Using Verilog HDL

Technical Project Report

RTL Design, Simulation, and Functional Verification using AMD Vivado

Author

Shivam Chaurasiya

B.Tech in Electronics and Communication Engineering

Project Type

Self-Directed Learning Project

Development Environment

Language	: Verilog HDL
EDA Tool	: AMD Vivado 2025.2
Verification	: Functional Simulation using Vivado Simulator
Target Design	: 4-Bit Processor
Architecture	: Single-Cycle Processor Architecture
Instruction Width	: 8-bit
Data Width	: 4-bit

July 2026

Contents

Abstract	5
1 Introduction	6
1.1 Project Overview	6
1.2 Project Objectives	6
1.3 Design Methodology	7
1.4 Processor Instruction Cycle	8
1.5 Design Specifications	9
2 Processor Architecture	10
2.1 Processor Overview	10
2.2 Processor Datapath	10
2.3 Processor Data Flow	12
2.4 Processor Features	12
3 Instruction Set Architecture	13
3.1 Instruction Format	13
3.2 Register Organization	13
3.3 Instruction Set	14
3.4 Instruction Memory Contents	14
3.5 Execution Example	14
4 Module Design and Verification	16
4.1 Program Counter	16
4.1.1 Overview	16
4.1.2 Operational Flow	16
4.1.3 RTL Schematic	17
4.1.4 Simulation Results	17
4.1.5 Observations	18
4.2 Instruction Memory	18
4.2.1 Overview	18
4.2.2 Operational Flow	18
4.2.3 Memory Organization	19
4.2.4 RTL Schematic	19
4.2.5 Simulation Results	19
4.2.6 Observations	19
4.3 Instruction Decoder	20
4.3.1 Overview	20
4.3.2 Operational Flow	20

4.3.3	Instruction Decoding Format	20
4.3.4	RTL Schematic	21
4.3.5	Simulation Results	22
4.3.6	Observations	22
4.4	Control Unit	22
4.4.1	Overview	22
4.4.2	Operational Flow	23
4.4.3	Generated Control Signals	23
4.4.4	RTL Schematic	23
4.4.5	Simulation Results	23
4.4.6	Observations	25
4.5	Register File	25
4.5.1	Overview	25
4.5.2	Register Organization	25
4.5.3	Operational Characteristics	26
4.5.4	RTL Schematic	26
4.5.5	Simulation Results	27
4.5.6	Observations	27
4.6	Arithmetic Logic Unit	27
4.6.1	Overview	27
4.6.2	ALU Architecture	28
4.6.3	Supported Operations	28
4.6.4	Status Flags	29
4.6.5	RTL Schematic	29
4.6.6	Simulation Results	29
4.6.7	Observations	30
5	Processor Integration and Functional Verification	31
5.1	Processor Integration	31
5.2	Integrated Processor RTL Architecture	31
5.3	Processor Execution Flow	32
5.4	Functional Verification	32
5.5	Instruction Execution Sequence	33
5.6	Verification Observations	33
6	Results and Discussion	35
6.1	Project Outcomes	35
6.2	Functional Verification Results	35
6.3	Supported Processor Features	35
6.4	Discussion	36
6.5	Future Improvements	36
7	Conclusion	37
	References	37

List of Figures

1.1	Overall Project Development Flow	7
1.2	Processor Instruction Execution Cycle	8
2.1	Detailed Processor Datapath Architecture	11
4.1	Program Counter Operational Flow	16
4.2	RTL Schematic of the Program Counter	17
4.3	Simulation Waveform of the Program Counter	17
4.4	Instruction Memory Operational Flow	18
4.5	RTL Schematic of the Instruction Memory	19
4.6	Simulation Waveform of the Instruction Memory	20
4.7	Instruction Decoder Operational Flow	21
4.8	RTL Schematic of the Instruction Decoder	21
4.9	Simulation Waveform of the Instruction Decoder	22
4.10	Control Unit Operational Flow	23
4.11	RTL Schematic of the Control Unit	24
4.12	Simulation Waveform of the Control Unit	24
4.13	Architecture of the 4×4 Register File	25
4.14	RTL Schematic of the Register File	26
4.15	Simulation Waveform of the Register File	27
4.16	Operational Flow of the Arithmetic Logic Unit	28
4.17	RTL Schematic of the Arithmetic Logic Unit	29
4.18	Simulation Waveform of the Arithmetic Logic Unit	30
5.1	RTL Schematic of the Complete 4-Bit Processor	32
5.2	Simulation Waveform of the Complete Processor	33

List of Tables

1.1	Processor Design Specifications	9
3.1	Instruction Format	13
3.2	Register Encoding	13
3.3	Instruction Set Architecture	14
3.4	Program Stored in Instruction Memory	14
4.1	Instruction Memory Specifications	19
4.2	Instruction Decoder Output Fields	21
4.3	Control Signals Generated by the Control Unit	23
4.4	Register Address Encoding	26
4.5	Supported ALU Operations	28
4.6	ALU Status Flags	29
5.1	Instruction Execution Sequence	33

Abstract

This report presents the design, implementation, and functional verification of a custom 4-bit single-cycle processor developed using Verilog Hardware Description Language (HDL). The processor architecture follows a modular design approach consisting of a Program Counter (PC), Instruction Memory (IM), Instruction Decoder (ID), Control Unit (CU), Register File (RF), and Arithmetic Logic Unit (ALU). The complete processor datapath enables instruction fetch, decode, execution, and write-back operations within a single clock cycle.

The processor executes 8-bit instructions composed of opcode and operand fields, allowing the implementation of arithmetic and logical operations through a simplified Instruction Set Architecture (ISA). Control signals required for processor operation are generated by the Control Unit based on the decoded instruction opcode. A 4-register register file with dual-read and single-write capability was designed to support operand storage and data movement within the processor.

Each module was individually verified using simulation testbenches and RTL analysis in AMD Vivado 2025.2 before being integrated into the complete processor architecture. RTL schematics and simulation waveforms were analyzed to validate the functionality of every subsystem and the overall processor operation. The final integrated design successfully demonstrated instruction execution, data transfer between modules, arithmetic and logical computations, and proper control signal generation.

The project provides practical exposure to digital processor design concepts including datapath development, instruction decoding, control signal generation, register organization, and hardware verification methodologies using industry-standard FPGA development tools.

Chapter 1

Introduction

1.1 Project Overview

Modern digital systems rely on processors to execute instructions, perform arithmetic and logical computations, and coordinate data movement between hardware components. A processor integrates multiple subsystems including an Arithmetic Logic Unit (ALU), register storage, control logic, and instruction memory to perform computation tasks.

This project presents the design and functional verification of a custom 4-bit single-cycle processor using Verilog Hardware Description Language (HDL). The processor architecture was developed using a modular design approach and verified using AMD Vivado 2025.2 simulation tools.

The processor supports arithmetic, logical, and data transfer instructions through a simple custom Instruction Set Architecture (ISA). The complete processor consists of the following modules:

- Program Counter (PC)
- Instruction Memory (IM)
- Instruction Decoder (ID)
- Control Unit (CU)
- Register File (RF)
- Arithmetic Logic Unit (ALU)

The integrated datapath performs instruction fetch, decode, execute, and write-back operations during processor execution.

1.2 Project Objectives

The primary objectives of this project are:

- Design a custom 4-bit processor architecture using Verilog HDL.
- Develop a simplified instruction set for arithmetic and logical operations.

- Implement modular processor components using synthesizable RTL design.
- Verify individual modules using simulation waveforms and RTL analysis.
- Integrate all modules to form a functional processor datapath.
- Validate processor operation through functional simulation in Vivado.

1.3 Design Methodology

The development of the processor followed a modular bottom-up design methodology where individual components were first designed and verified independently before system integration.

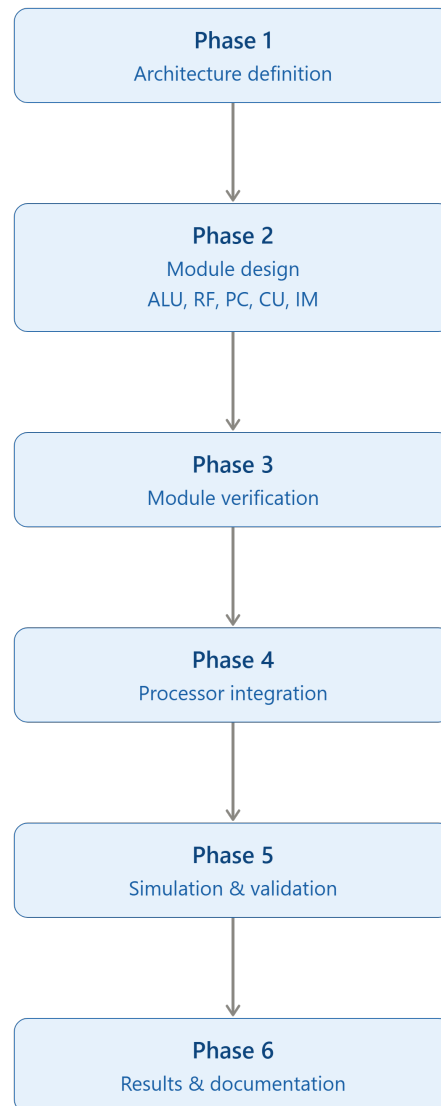


Figure 1.1: Overall Project Development Flow

The design process began with the definition of processor specifications and instruction format, followed by the implementation of individual datapath modules and finally complete processor integration and verification.

1.4 Processor Instruction Cycle

The processor follows a simplified single-cycle execution model consisting of instruction fetch, instruction decode, execution, write-back, and program counter update stages.

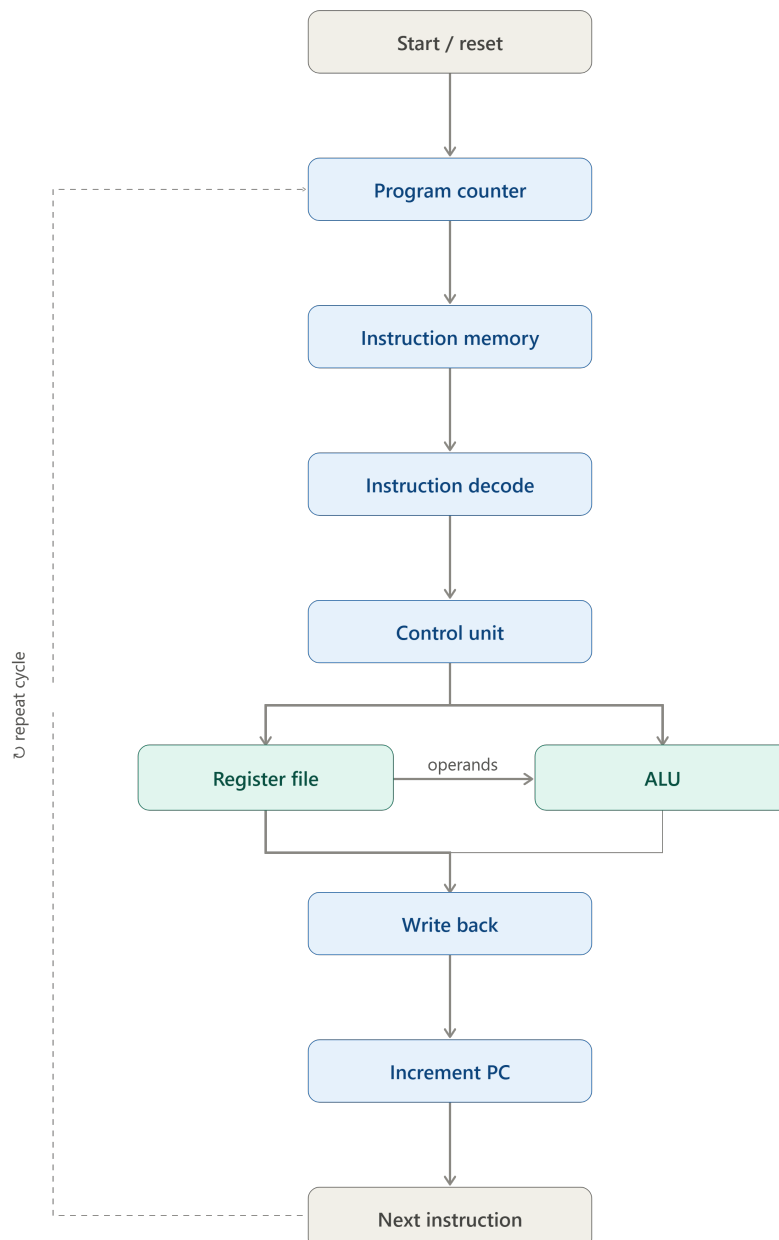


Figure 1.2: Processor Instruction Execution Cycle

During each instruction cycle, the Program Counter provides the address of the cur-

rent instruction to the Instruction Memory. The fetched instruction is decoded by the Instruction Decoder and Control Unit, which generate the control signals required for execution. Operands are obtained from the Register File, processed by the ALU, and the result is written back to the destination register before advancing to the next instruction.

1.5 Design Specifications

The design specifications of the implemented processor are summarized in Table 1.1.

Table 1.1: Processor Design Specifications

Parameter	Specification
Processor Type	Single-Cycle Processor
Data Width	4-bit
Instruction Width	8-bit
Address Width	4-bit
Register Count	4 Registers
Instruction Memory Size	16×8 bits
ALU Operations	9
HDL Used	Verilog HDL
EDA Tool	AMD Vivado 2025.2
Verification Method	Functional Simulation

Chapter 2

Processor Architecture

2.1 Processor Overview

The implemented processor follows a single-cycle architecture in which instruction fetch, decode, execution, and write-back operations are completed during a single instruction cycle. The processor was designed using a modular datapath approach to simplify design, verification, and future extensibility.

The processor consists of six major hardware modules:

- Program Counter (PC)
- Instruction Memory (IM)
- Instruction Decoder (ID)
- Control Unit (CU)
- Register File (RF)
- Arithmetic Logic Unit (ALU)

These modules interact through a common datapath to perform instruction execution and data processing.

2.2 Processor Datapath

Figure [2.1](#) illustrates the detailed datapath architecture of the implemented 4-bit processor.

The Program Counter generates the instruction address which is supplied to the Instruction Memory. The fetched instruction is forwarded to the Instruction Decoder where the opcode and register fields are extracted.

The decoded opcode is provided to the Control Unit, which generates the control signals required for processor operation, including ALU operation selection, register write enable, and program counter control signals.

The Register File stores temporary operands and intermediate computation results. Two operands are simultaneously read from the register file and supplied to the ALU for execution.

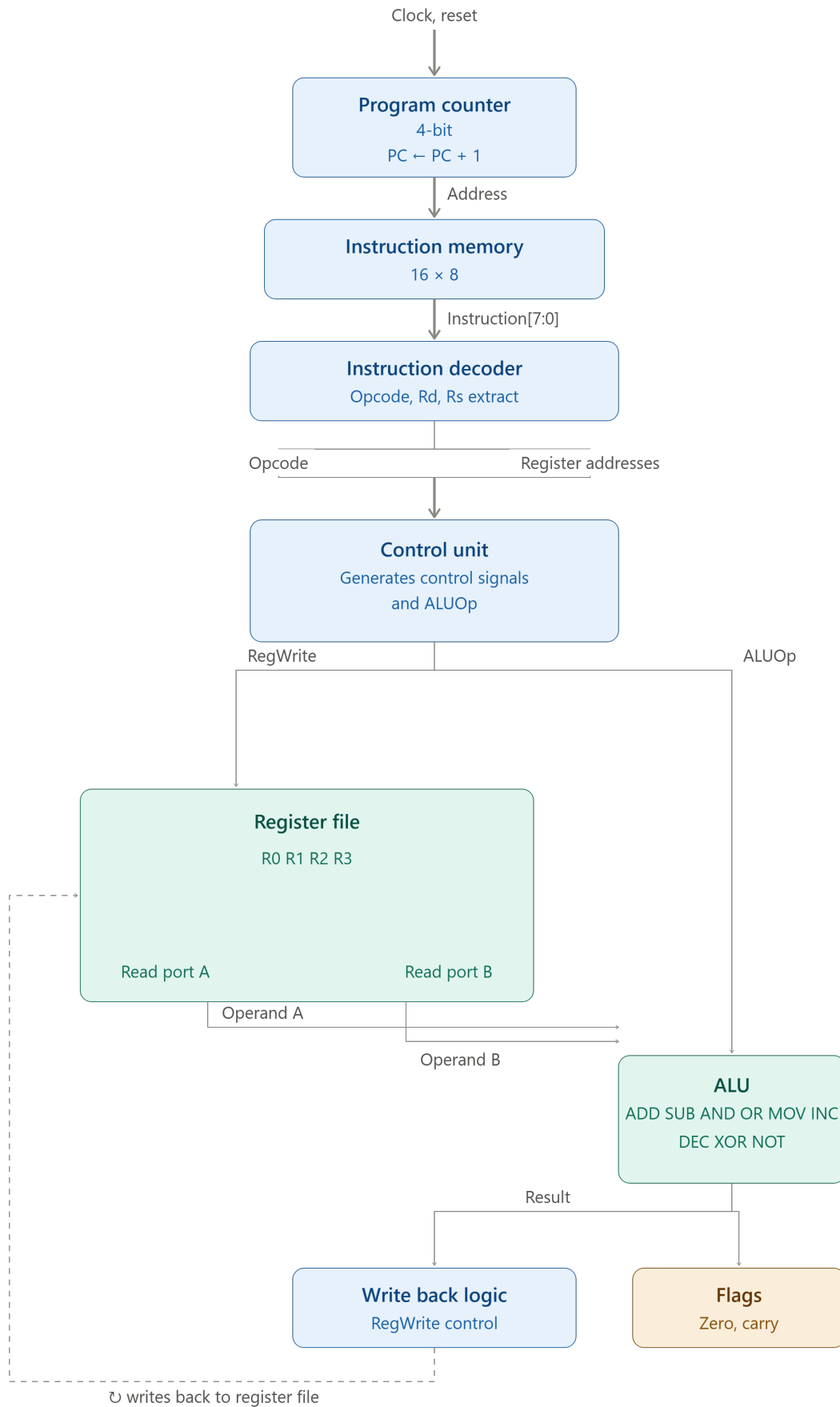


Figure 2.1: Detailed Processor Datapath Architecture

The ALU performs arithmetic and logical operations according to the control signals generated by the Control Unit. The computation result is written back into the destination register to complete the instruction execution process.

2.3 Processor Data Flow

The overall processor operation follows the sequence shown below:

1. Program Counter generates the instruction address.
2. Instruction Memory fetches the corresponding instruction.
3. Instruction Decoder extracts opcode and register information.
4. Control Unit generates required control signals.
5. Register File supplies operands to the ALU.
6. ALU performs the requested operation.
7. The result is written back into the destination register.
8. Program Counter advances to the next instruction.

This sequence forms the fundamental instruction execution cycle of the processor.

2.4 Processor Features

The major features of the implemented processor are summarized below:

- Custom 8-bit instruction format.
- Single-cycle instruction execution.
- Four general purpose 4-bit registers.
- Arithmetic and logical instruction support.
- Modular RTL design methodology.
- Functional verification using simulation waveforms.
- Scalable architecture for future extensions.

Chapter 3

Instruction Set Architecture

3.1 Instruction Format

The implemented processor uses an 8-bit instruction format to represent arithmetic, logical, and data transfer operations. Each instruction consists of an opcode field and register address fields as shown below.

Table 3.1: Instruction Format

Bits [7:4]	Bits [3:2]	Bits [1:0]
Opcode	Destination Register (Rd)	Source Register (Rs)

The opcode determines the operation to be executed by the processor, while the register fields specify the source and destination operands stored in the Register File.

- Opcode Width = 4 bits
- Destination Register Width = 2 bits
- Source Register Width = 2 bits
- Total Instruction Width = 8 bits

3.2 Register Organization

The processor contains four general-purpose 4-bit registers used for temporary data storage and arithmetic operations.

Table 3.2: Register Encoding

Register Address	Register Name
00	R0
01	R1
10	R2
11	R3

The Register File supports two simultaneous read operations and one write operation during each instruction cycle.

3.3 Instruction Set

The processor supports arithmetic, logical, and data movement instructions as shown in Table 3.3.

Table 3.3: Instruction Set Architecture

Opcode	Instruction	Operation Performed
0000	ADD	$Rd \leftarrow Rd + Rs$
0001	SUB	$Rd \leftarrow Rd - Rs$
0010	AND	$Rd \leftarrow Rd \text{ AND } Rs$
0011	OR	$Rd \leftarrow Rd \text{ OR } Rs$
0100	XOR	$Rd \leftarrow Rd \text{ XOR } Rs$
0101	MOV	$Rd \leftarrow Rs$
0110	INC	$Rd \leftarrow Rd + 1$
0111	DEC	$Rd \leftarrow Rd - 1$
1000	NOT	$Rd \leftarrow \text{NOT}(Rd)$
1111	HLT	Halt Processor Execution

3.4 Instruction Memory Contents

The instruction memory stores the program to be executed by the processor. The example program used for processor verification is shown below.

Table 3.4: Program Stored in Instruction Memory

Address	Instruction	Operation
0	0x51	MOV R0, R1
1	0x02	ADD R0, R2
2	0x13	SUB R0, R3
3	0x60	INC R0
4	0xF0	HLT

This instruction sequence was used to verify processor functionality during simulation and integration testing.

3.5 Execution Example

Assuming the initial register contents are:

$$R0 = 5, \quad R1 = 2, \quad R2 = 3, \quad R3 = 1$$

the processor executes the following sequence:

1. Transfer the value of R1 into R0.
2. Add the contents of R2 to R0.

3. Subtract the contents of R3 from R0.
4. Increment R0 by one.
5. Halt processor execution.

The successful execution of this instruction sequence verifies the correct operation of the processor datapath and control logic.

Chapter 4

Module Design and Verification

4.1 Program Counter

4.1.1 Overview

The Program Counter (PC) is responsible for generating the address of the instruction to be executed by the processor. It stores the address of the current instruction and increments sequentially after every instruction execution cycle.

The Program Counter supports the following operations:

- Reset operation to initialize the processor execution sequence.
- Sequential address increment after instruction execution.
- Program execution halt through the `pc_enable` control signal.

The Program Counter outputs a 4-bit address which is supplied to the Instruction Memory for instruction fetching.

4.1.2 Operational Flow

The internal operation of the Program Counter is illustrated in Figure 4.1.

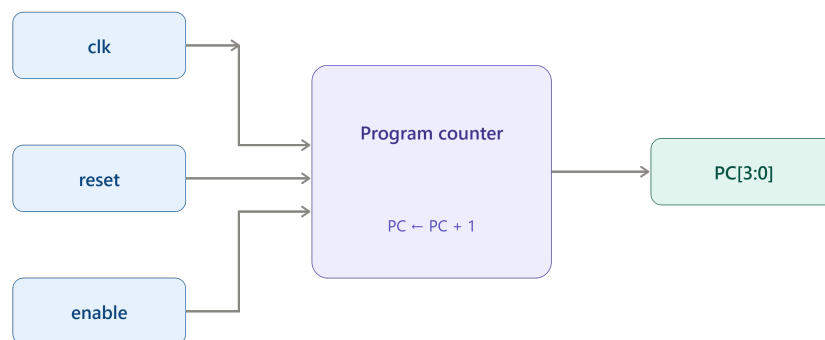


Figure 4.1: Program Counter Operational Flow

During reset, the Program Counter is initialized to zero. During normal operation, the Program Counter increments by one at every positive clock edge when the `pc_enable` signal is asserted.

4.1.3 RTL Schematic

The RTL schematic generated by Vivado after elaboration is shown in Figure 4.2.

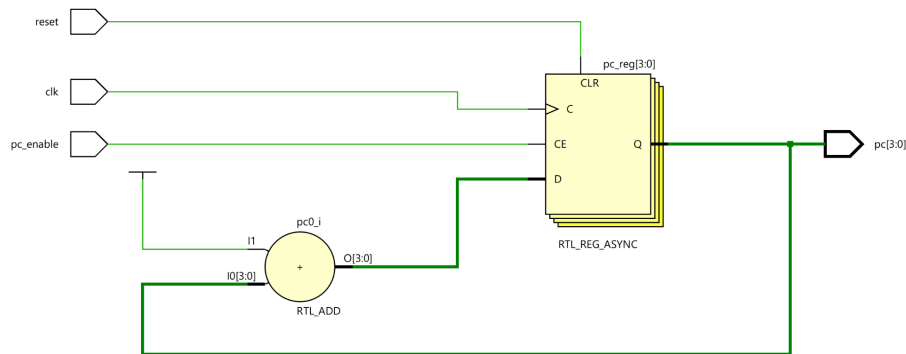


Figure 4.2: RTL Schematic of the Program Counter

The RTL schematic confirms the implementation of a synchronous 4-bit counter with reset and enable functionality.

4.1.4 Simulation Results

The simulation waveform used to verify the Program Counter functionality is shown in Figure 4.3.

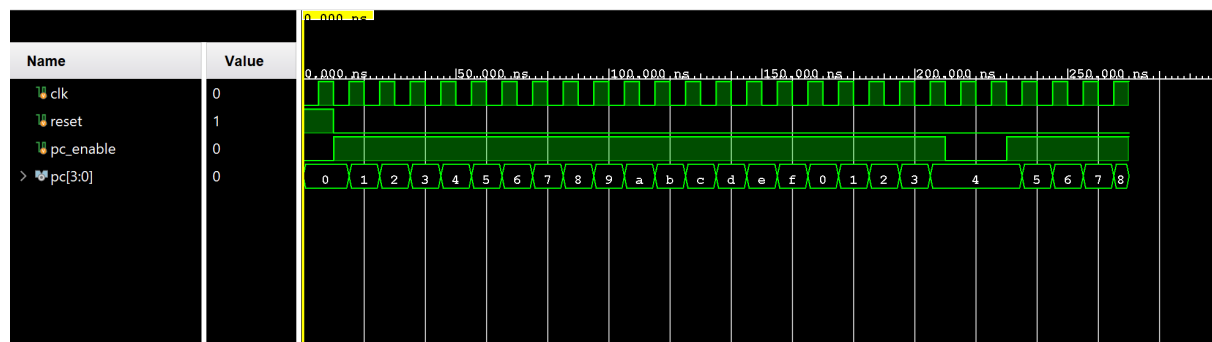


Figure 4.3: Simulation Waveform of the Program Counter

The waveform verifies that the Program Counter initializes correctly during reset and increments sequentially on every clock cycle when enabled. The counter stops incrementing when the processor executes the halt instruction and the `pc_enable` signal is deasserted.

4.1.5 Observations

The Program Counter successfully generated sequential instruction addresses and correctly responded to reset and halt conditions during simulation, thereby enabling reliable instruction fetch operations within the processor datapath.

4.2 Instruction Memory

4.2.1 Overview

The Instruction Memory (IM) stores the program instructions executed by the processor. It is implemented as a read-only memory (ROM) consisting of sixteen 8-bit memory locations, allowing the processor to store up to sixteen instructions.

The Program Counter supplies the memory address, and the corresponding instruction is fetched and forwarded to the Instruction Decoder for further processing.

The Instruction Memory performs the following tasks:

- Stores the processor program.
- Provides instruction fetch functionality.
- Supplies instructions to the decoder based on the Program Counter value.

4.2.2 Operational Flow

The operational flow of the Instruction Memory is shown in Figure 4.4.

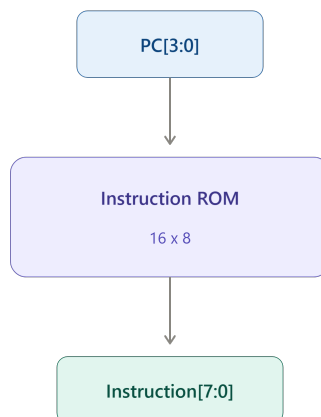


Figure 4.4: Instruction Memory Operational Flow

The Program Counter generates an instruction address which is supplied to the Instruction Memory. The corresponding instruction stored at that location is fetched and delivered to the Instruction Decoder.

4.2.3 Memory Organization

The implemented Instruction Memory contains 16 memory locations with each location storing an 8-bit instruction.

Table 4.1: Instruction Memory Specifications

Parameter	Value
Memory Depth	16 Locations
Instruction Width	8 Bits
Address Width	4 Bits
Memory Type	ROM
Access Method	Combinational Read

The instruction memory stores the example verification program discussed in the Instruction Set Architecture chapter.

4.2.4 RTL Schematic

The RTL schematic generated by Vivado for the Instruction Memory is shown in Figure 4.5.

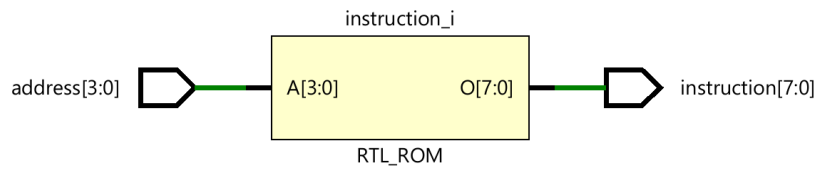


Figure 4.5: RTL Schematic of the Instruction Memory

The RTL schematic confirms the implementation of a ROM structure with address input and instruction output ports.

4.2.5 Simulation Results

The simulation waveform used to verify the Instruction Memory is shown in Figure 4.6.

The waveform verifies that different instruction addresses generated by the Program Counter correctly retrieve the corresponding instructions stored in memory.

4.2.6 Observations

The Instruction Memory successfully provided correct instruction fetch functionality during simulation. The fetched instructions matched the programmed contents of memory, confirming correct address decoding and data retrieval operations.

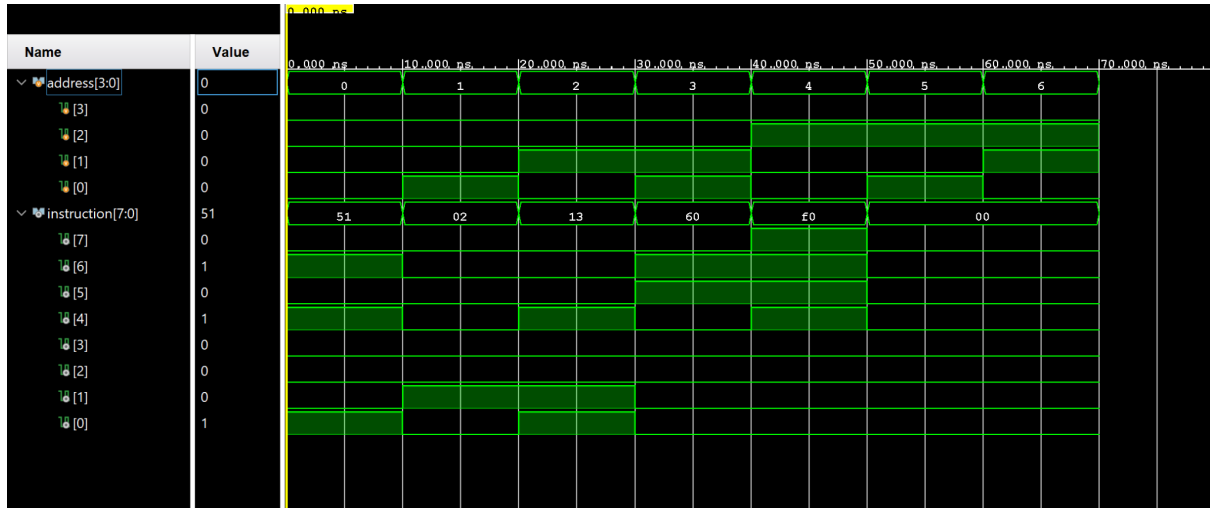


Figure 4.6: Simulation Waveform of the Instruction Memory

4.3 Instruction Decoder

4.3.1 Overview

The Instruction Decoder (ID) is responsible for interpreting the instruction fetched from the Instruction Memory and extracting the information required for processor execution. The decoder separates the instruction into opcode and register address fields which are subsequently used by the Control Unit and Register File.

The Instruction Decoder performs the following functions:

- Extracts the opcode field from the instruction.
- Determines the destination register address.
- Determines the source register address.
- Supplies decoded information to the Control Unit and Register File.

The implemented processor uses an 8-bit instruction format consisting of a 4-bit opcode and two 2-bit register addresses.

4.3.2 Operational Flow

The internal operation of the Instruction Decoder is illustrated in Figure 4.7.

The fetched instruction is received from the Instruction Memory and divided into three fields: opcode, destination register address, and source register address.

4.3.3 Instruction Decoding Format

The instruction fields extracted by the decoder are summarized in Table 4.2.

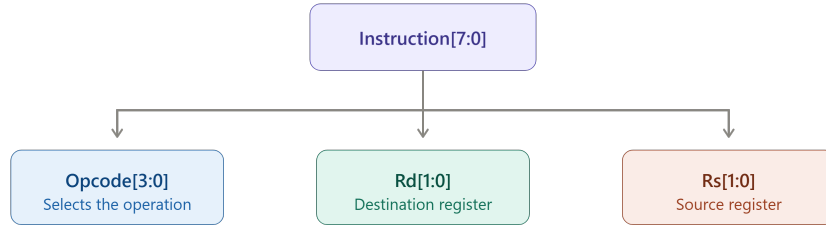


Figure 4.7: Instruction Decoder Operational Flow

Table 4.2: Instruction Decoder Output Fields

Instruction Bits	Field Name	Width
[7:4]	Opcode	4 bits
[3:2]	Destination Register (Rd)	2 bits
[1:0]	Source Register (Rs)	2 bits

The opcode field is forwarded to the Control Unit while the register addresses are supplied to the Register File for operand access.

4.3.4 RTL Schematic

The RTL schematic generated by Vivado for the Instruction Decoder is shown in Figure 4.8.

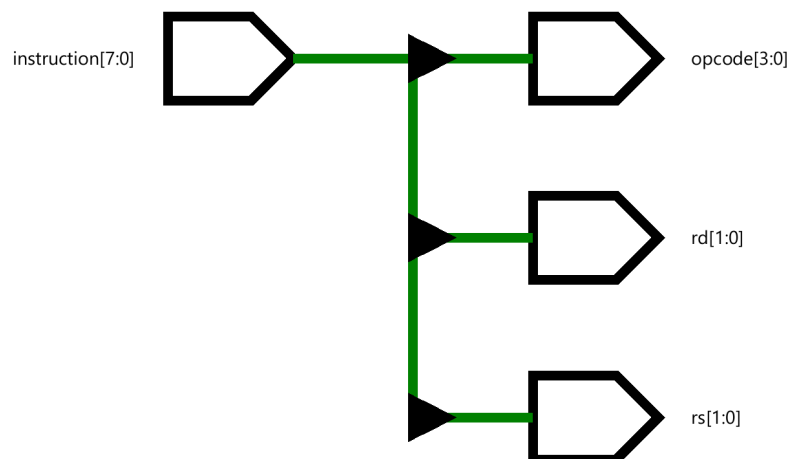


Figure 4.8: RTL Schematic of the Instruction Decoder

The RTL schematic confirms that the decoder simply partitions the instruction bits into their corresponding functional fields.

4.3.5 Simulation Results

The simulation waveform used to verify the Instruction Decoder is shown in Figure 4.9.

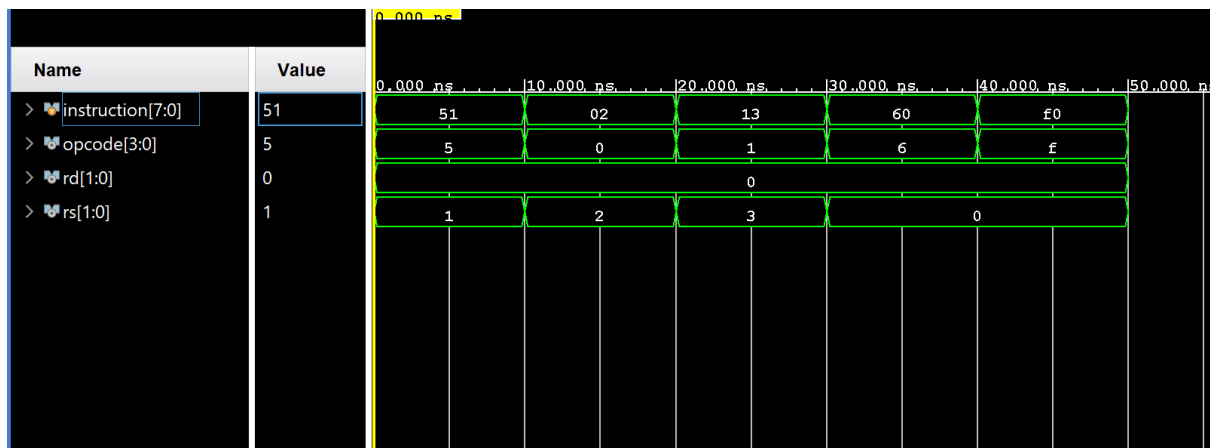


Figure 4.9: Simulation Waveform of the Instruction Decoder

The waveform verifies that the opcode, destination register, and source register fields are correctly extracted for different instruction inputs.

4.3.6 Observations

The Instruction Decoder successfully extracted all instruction fields and correctly supplied the decoded information to the Control Unit and Register File during simulation. The module forms an essential interface between instruction fetch and execution stages of the processor datapath.

4.4 Control Unit

4.4.1 Overview

The Control Unit (CU) is responsible for generating the control signals required for processor operation. It receives the decoded opcode from the Instruction Decoder and determines the appropriate ALU operation and control signals needed to execute the instruction.

The Control Unit performs the following functions:

- Generates ALU operation selection signals.
- Controls register write operations.
- Controls Program Counter updates.
- Detects the halt instruction and stops processor execution.

The generated control signals coordinate the operation of all datapath components and ensure correct instruction execution.

4.4.2 Operational Flow

The operation of the Control Unit is illustrated in Figure 4.10.

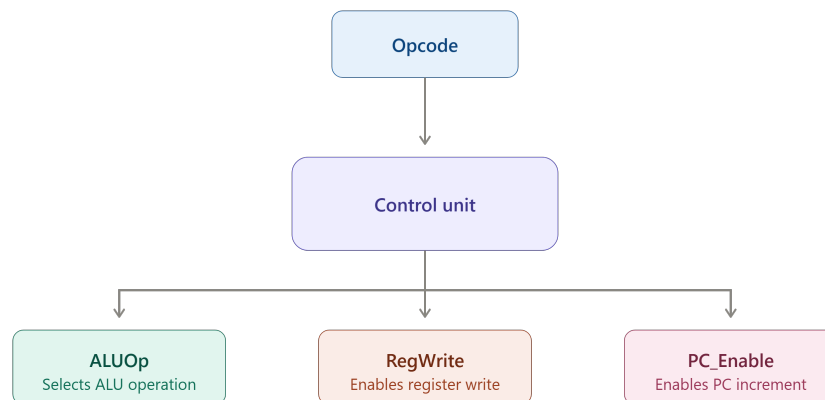


Figure 4.10: Control Unit Operational Flow

The opcode received from the Instruction Decoder is analyzed and translated into corresponding control signals for the ALU, Register File, and Program Counter.

4.4.3 Generated Control Signals

The Control Unit generates the signals summarized in Table 4.3.

Table 4.3: Control Signals Generated by the Control Unit

Signal	Width	Function
ALU_Op	4 bits	Selects ALU operation
RegWrite	1 bit	Enables register write operation
PC_Enable	1 bit	Controls Program Counter increment

The ALU operation signal determines the arithmetic or logical function to be performed, while the remaining signals control data movement and instruction sequencing.

4.4.4 RTL Schematic

The RTL schematic generated by Vivado for the Control Unit is shown in Figure 4.11.

The RTL schematic verifies the combinational logic used for opcode decoding and control signal generation.

4.4.5 Simulation Results

The simulation waveform used to verify the Control Unit functionality is shown in Figure 4.12.

The waveform confirms that the Control Unit correctly generates the required control signals for each opcode. The halt instruction disables the Program Counter and prevents further instruction execution.

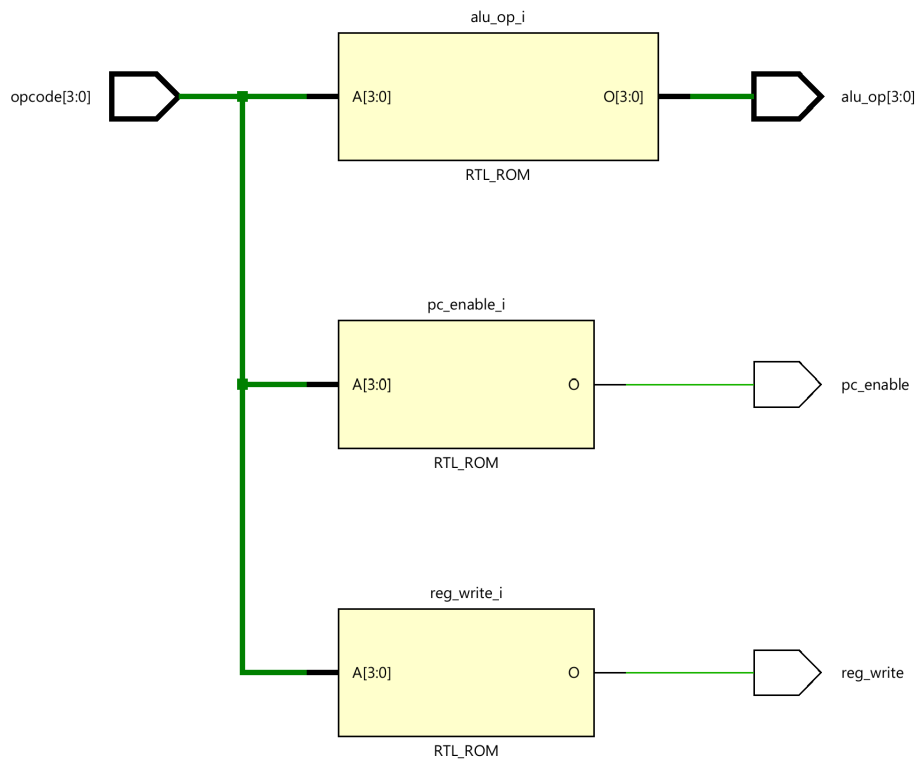


Figure 4.11: RTL Schematic of the Control Unit



Figure 4.12: Simulation Waveform of the Control Unit

4.4.6 Observations

The Control Unit successfully decoded all supported instructions and generated the corresponding control signals required for processor operation. The module correctly managed ALU operation selection, register write control, and processor halt functionality.

4.5 Register File

4.5.1 Overview

The Register File (RF) serves as the primary storage unit of the processor and is responsible for holding operands, intermediate values, and computation results during instruction execution. The implemented register file consists of four general-purpose 4-bit registers.

The Register File supports two simultaneous read operations and one write operation, enabling efficient data transfer between the processor datapath components.

The Register File performs the following functions:

- Stores operands required for ALU operations.
- Provides simultaneous access to two source operands.
- Stores computation results generated by the ALU.
- Supports register-to-register data movement operations.

4.5.2 Register Organization

The implemented register file consists of four 4-bit registers as illustrated in Figure 4.13.

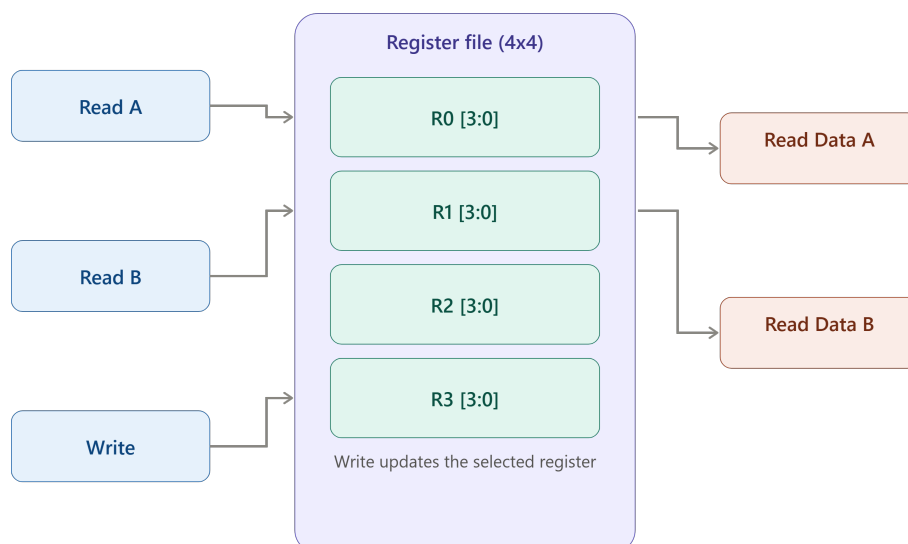


Figure 4.13: Architecture of the 4×4 Register File

The register addresses are encoded using two bits as shown below:

Table 4.4: Register Address Encoding

Address	Register
00	R0
01	R1
10	R2
11	R3

4.5.3 Operational Characteristics

The Register File provides:

- Two independent read ports.
- One synchronous write port.
- Register initialization through reset.
- Support for data transfer and ALU result storage.

The two read ports allow the processor to fetch two operands simultaneously, while the write port stores the ALU result back into the destination register.

4.5.4 RTL Schematic

The RTL schematic generated by Vivado for the Register File is shown in Figure 4.14.

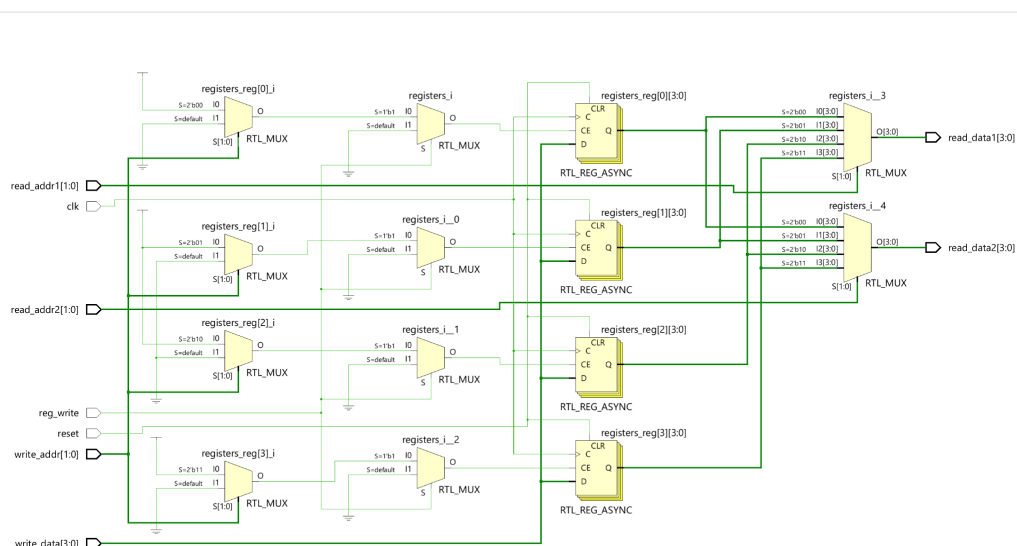


Figure 4.14: RTL Schematic of the Register File

The RTL schematic confirms the implementation of four storage registers along with dual read ports and a single write port.

4.5.5 Simulation Results

The simulation waveform used to verify the Register File functionality is shown in Figure 4.15.

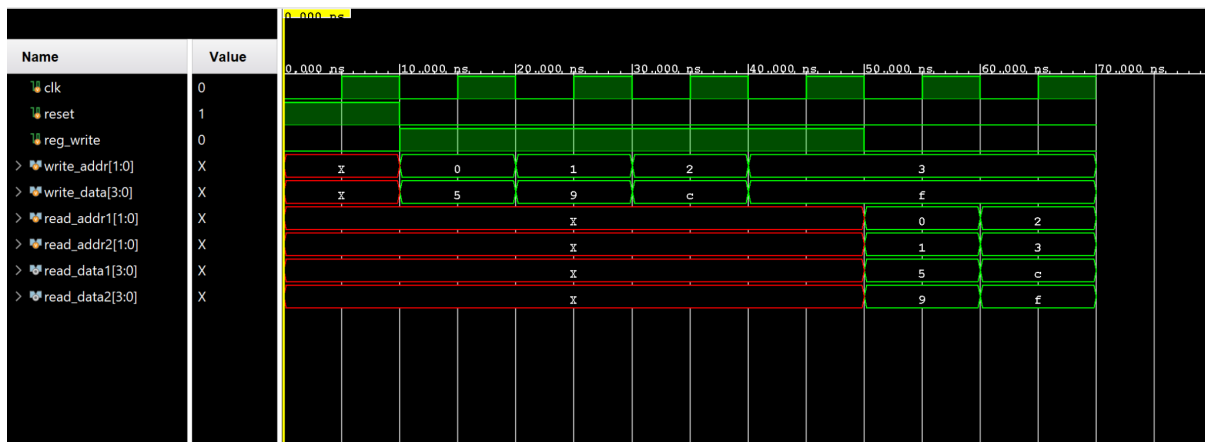


Figure 4.15: Simulation Waveform of the Register File

The waveform verifies the correct operation of register read and write functionality. Data written into a destination register is correctly stored and subsequently retrieved through the read ports.

4.5.6 Observations

The Register File successfully provided dual operand access and reliable result storage throughout processor operation. The module correctly supported simultaneous read operations and synchronous write-back of ALU results, thereby enabling efficient datapath operation.

4.6 Arithmetic Logic Unit

4.6.1 Overview

The Arithmetic Logic Unit (ALU) is the computational core of the processor and is responsible for performing arithmetic, logical, and data transfer operations. The ALU receives two 4-bit operands from the Register File along with an operation select signal generated by the Control Unit.

Based on the selected operation, the ALU performs the required computation and produces a 4-bit result along with status flags for carry and zero conditions.

The ALU performs the following functions:

- Arithmetic operations.
- Logical operations.
- Data transfer operations.
- Generation of status flags.

4.6.2 ALU Architecture

The operational architecture of the ALU is illustrated in Figure 4.16.

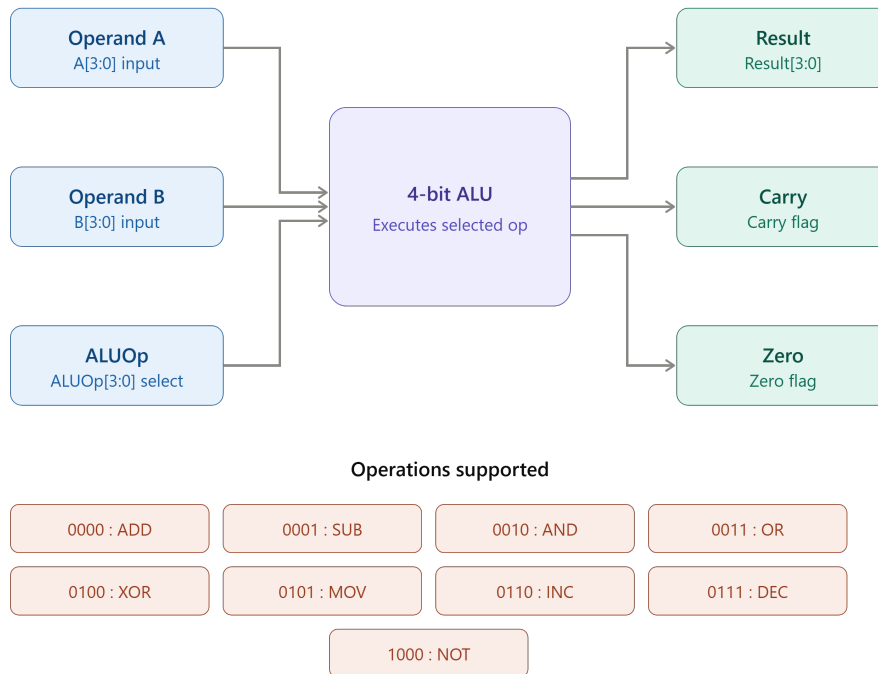


Figure 4.16: Operational Flow of the Arithmetic Logic Unit

The ALU receives operands from the Register File and executes the operation specified by the ALU operation signal generated by the Control Unit.

4.6.3 Supported Operations

The implemented ALU supports the operations listed in Table 4.5.

Table 4.5: Supported ALU Operations

Opcode	Operation	Description
0000	ADD	Addition
0001	SUB	Subtraction
0010	AND	Bitwise AND
0011	OR	Bitwise OR
0100	XOR	Bitwise XOR
0101	MOV	Data Transfer
0110	INC	Increment Operand A
0111	DEC	Decrement Operand A
1000	NOT	Bitwise Inversion

4.6.4 Status Flags

The ALU generates two status flags to indicate computation results.

Table 4.6: ALU Status Flags

Flag	Function
Carry Zero	Indicates arithmetic overflow
Zero	Indicates result equal to zero

These flags provide additional information regarding the result of arithmetic operations and can be extended in future versions of the processor to support conditional branching instructions.

4.6.5 RTL Schematic

The RTL schematic generated by Vivado for the ALU is shown in Figure 4.17.

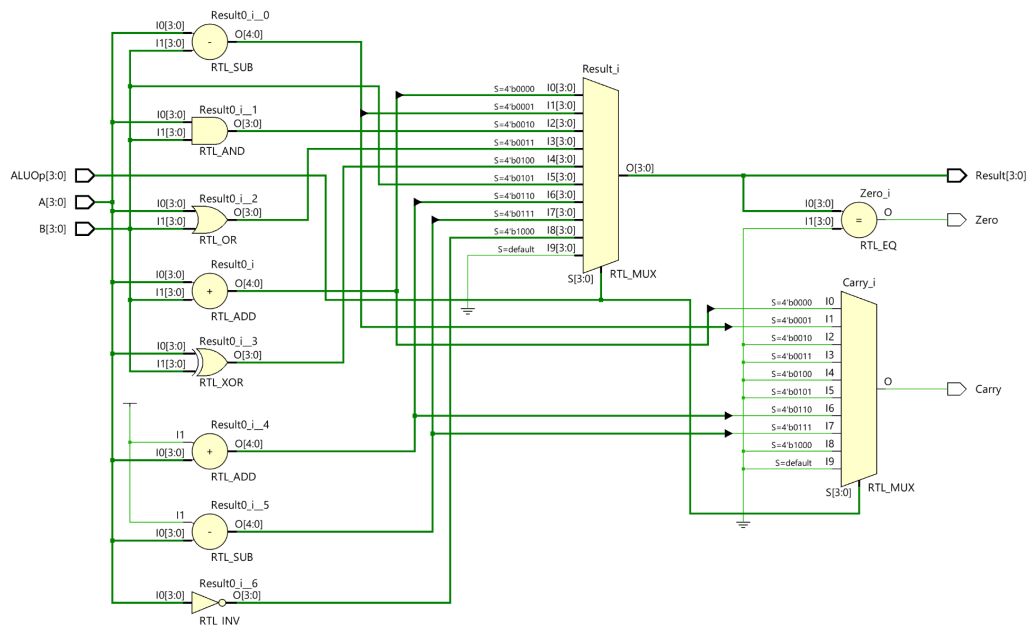


Figure 4.17: RTL Schematic of the Arithmetic Logic Unit

The RTL schematic confirms the implementation of combinational logic responsible for arithmetic and logical computations.

4.6.6 Simulation Results

The simulation waveform used to verify the ALU functionality is shown in Figure 4.18.

The waveform verifies the correct execution of all supported operations including arithmetic computations, logical manipulations, and data transfer instructions. The generated status flags were also validated during simulation.

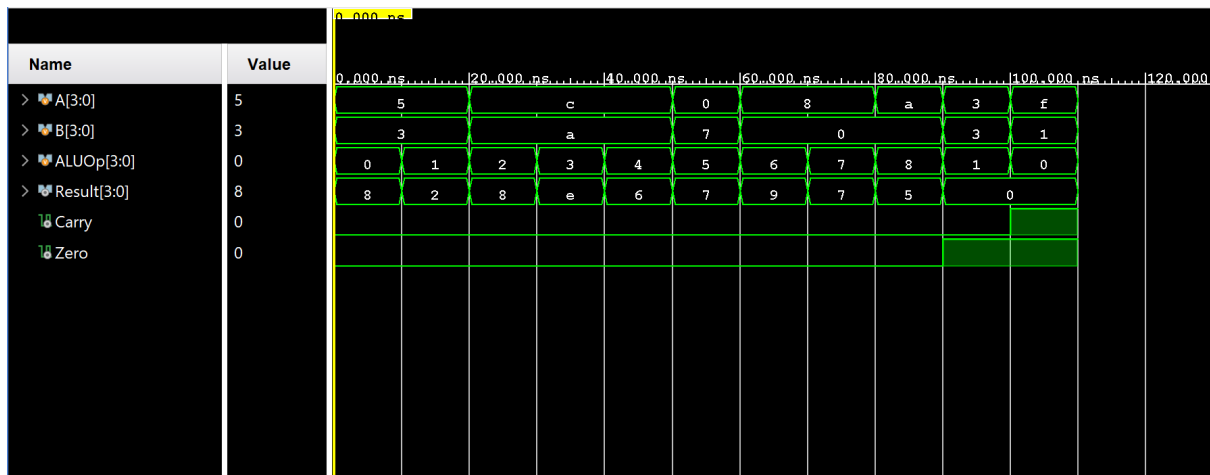


Figure 4.18: Simulation Waveform of the Arithmetic Logic Unit

4.6.7 Observations

The Arithmetic Logic Unit successfully executed all supported instructions and generated correct results for each operation. The ALU formed the computational backbone of the processor and correctly interacted with the Control Unit and Register File during processor execution.

Chapter 5

Processor Integration and Functional Verification

5.1 Processor Integration

After the successful verification of individual hardware modules, all components were integrated to form the complete 4-bit processor architecture. The integration process involved connecting the datapath and control path components to enable instruction fetch, decode, execution, and write-back operations.

The integrated processor consists of the following modules:

- Program Counter (PC)
- Instruction Memory (IM)
- Instruction Decoder (ID)
- Control Unit (CU)
- Register File (RF)
- Arithmetic Logic Unit (ALU)

These modules communicate through internal buses and control signals to execute instructions stored in the instruction memory.

5.2 Integrated Processor RTL Architecture

The complete RTL schematic generated by Vivado after elaboration is shown in Figure 5.1.

The RTL schematic verifies the successful integration of all processor modules and confirms the correctness of datapath connections and control signal routing.

The Program Counter generates the instruction address, which is used by the Instruction Memory to fetch the corresponding instruction. The Instruction Decoder extracts the instruction fields and forwards them to the Control Unit and Register File. The Control Unit generates the required control signals while the Register File supplies operands to the Arithmetic Logic Unit for execution.

The computation result generated by the ALU is written back into the Register File to complete the instruction execution cycle.

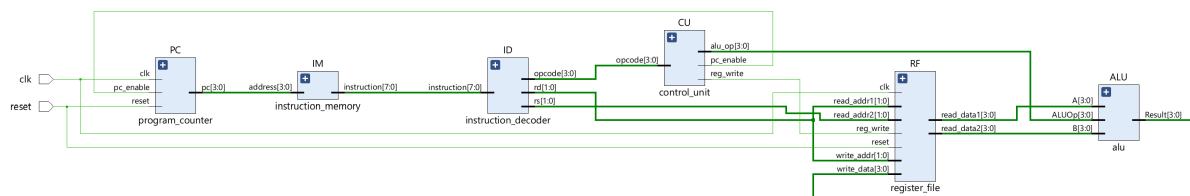


Figure 5.1: RTL Schematic of the Complete 4-Bit Processor

5.3 Processor Execution Flow

The integrated processor follows the execution sequence shown below:

1. Program Counter generates instruction address.
2. Instruction Memory fetches instruction.
3. Instruction Decoder extracts instruction fields.
4. Control Unit generates control signals.
5. Register File provides operands.
6. ALU performs computation.
7. Result is written back into the destination register.
8. Program Counter advances to the next instruction.

This sequence forms the fetch-decode-execute-writeback cycle of the processor.

5.4 Functional Verification

The processor functionality was verified using simulation in AMD Vivado 2025.2. The simulation waveform of the complete processor is shown in Figure 5.2.

The waveform verifies correct interaction between all processor modules during instruction execution. The Program Counter advances sequentially, instructions are fetched correctly from memory, control signals are generated appropriately, and the ALU performs the requested computations.

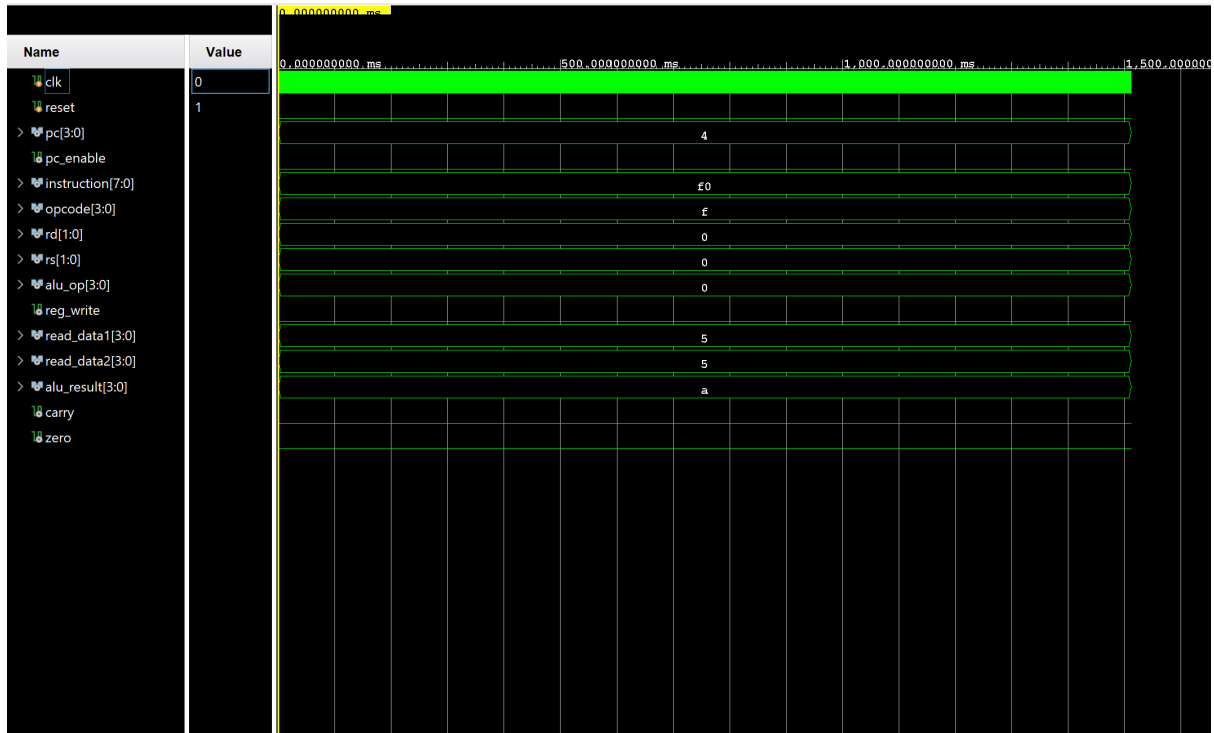


Figure 5.2: Simulation Waveform of the Complete Processor

5.5 Instruction Execution Sequence

The instruction sequence used for processor verification is summarized in Table 5.1.

Table 5.1: Instruction Execution Sequence

Program Counter	Instruction	Operation
0	MOV R0, R1	Transfer contents of R1 to R0
1	ADD R0, R2	Add R2 to R0
2	SUB R0, R3	Subtract R3 from R0
3	INC R0	Increment R0 by one
4	HLT	Halt processor execution

The successful execution of this instruction sequence confirms the correctness of the processor datapath and control logic.

5.6 Verification Observations

The following observations were made during simulation:

- The Program Counter incremented correctly after each instruction.
- Instructions were fetched correctly from the Instruction Memory.
- The Instruction Decoder extracted instruction fields accurately.
- The Control Unit generated correct control signals for each opcode.

- The Register File supplied operands and stored results correctly.
- The ALU executed arithmetic and logical operations successfully.
- The processor halted correctly upon execution of the HLT instruction.

The simulation results demonstrate successful integration and operation of the complete processor architecture.

Chapter 6

Results and Discussion

6.1 Project Outcomes

The design and verification of the custom 4-bit processor were successfully completed using Verilog HDL and AMD Vivado 2025.2. Individual hardware modules were first developed and verified independently before being integrated into the complete processor architecture.

Simulation results confirmed the correct functionality of all major processor components including the Program Counter, Instruction Memory, Instruction Decoder, Control Unit, Register File, and Arithmetic Logic Unit.

6.2 Functional Verification Results

The processor successfully demonstrated the following capabilities:

- Sequential instruction fetching using the Program Counter.
- Instruction retrieval from Instruction Memory.
- Correct extraction of instruction fields by the Instruction Decoder.
- Proper generation of control signals by the Control Unit.
- Dual operand access and result storage through the Register File.
- Successful execution of arithmetic and logical operations by the ALU.
- Correct write-back of computation results into destination registers.
- Proper processor termination upon execution of the halt instruction.

6.3 Supported Processor Features

The implemented processor provides the following features:

- 4-bit datapath architecture.
- 8-bit custom instruction format.

- Four general-purpose registers.
- Arithmetic instruction support.
- Logical instruction support.
- Data transfer instruction support.
- Single-cycle instruction execution.
- Modular and scalable design architecture.

6.4 Discussion

The modular design methodology significantly simplified the development and verification process. Independent testing of each module enabled easier debugging and ensured reliable integration at the processor level.

The implemented architecture successfully demonstrates the fundamental concepts of processor design including datapath organization, instruction decoding, control signal generation, register management, and arithmetic computation.

Although the processor is intentionally simple, the architecture can be extended to support additional features such as branch instructions, data memory, stack operations, and pipelined execution.

6.5 Future Improvements

Several enhancements can be incorporated into future versions of the processor:

- Addition of data memory support.
- Implementation of branch and jump instructions.
- Introduction of stack operations.
- Expansion to an 8-bit datapath.
- Support for interrupt handling.
- Pipeline implementation for improved performance.
- FPGA deployment on physical hardware platforms.

Chapter 7

Conclusion

A custom 4-bit single-cycle processor was successfully designed, implemented, and functionally verified using Verilog Hardware Description Language and AMD Vivado 2025.2. The processor architecture consisted of a Program Counter, Instruction Memory, Instruction Decoder, Control Unit, Register File, and Arithmetic Logic Unit integrated through a common datapath.

The processor successfully executed arithmetic, logical, and data transfer instructions using a custom instruction set architecture. Individual module verification through simulation waveforms and RTL schematics ensured reliable functionality prior to processor integration.

The final integrated processor demonstrated correct instruction fetch, decode, execute, and write-back operations along with proper processor termination through the halt instruction.

This project provided practical understanding of processor datapath design, control signal generation, instruction execution mechanisms, and digital hardware verification methodologies. The modular architecture also provides a suitable foundation for future enhancements and more advanced processor designs.

Bibliography

- [1] M. Morris Mano and Michael D. Ciletti, *Digital Design*, 5th Edition, Pearson Education, 2013.
- [2] David Money Harris and Sarah L. Harris, *Digital Design and Computer Architecture*, 2nd Edition, Morgan Kaufmann Publishers, 2012.
- [3] David A. Patterson and John L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, 5th Edition, Morgan Kaufmann Publishers, 2013.
- [4] Samir Palnitkar, *Verilog HDL: A Guide to Digital Design and Synthesis*, 2nd Edition, Pearson Education, 2003.
- [5] AMD, *Vivado Design Suite User Guide*, Version 2025.2, Advanced Micro Devices Inc.